

additive-model

2026-06-18

KEY RESULTS

I have found a model that feels more principled and basically agrees with SCEPTRE, except it also does better in the situations where SCEPTRE loses precision with heavy tailed guides.

So at this point I do have a model that uniformly improves upon SCEPTRE in all my simulation settings, as well as real data!

It still has a rather questionable decision in it, to do the γ vs e^γ difference, but I will give some mathematical justification. And it works...

1. The problem

SCEPTRE uses $e^{o_i} + \gamma$ as the perturbed mean, when it should be $e^{o_i + \gamma}$, and yet performance mysteriously is far better than if we actually did the correct model.

1.2 Using a summed model to replicate SCEPTRE's likelihood ratio

We can get this form for the mean via

$$Y_i \sim \text{Pois}(e^{o_i}) + \text{Ber}(\pi)\text{Pois}(e^{o_i} + e^\delta).$$

This is a special case of the Cellbender model that I discussed on Monday, with no endogenous noise in the NP distribution, and Poisson instead of NB for the perturbed component. So this is a very simple form of a reasonable model that appears elsewhere in the literature.

We then get a likelihood ratio of

$$\frac{\text{Pois}(y|e^o + e^\delta)}{\text{Pois}(y|e^o)}$$

which is the same additive form that SCEPTRE accidentally uses.

So that's all fine and good... except the MLE of e^δ will be very large, since it's on the response scale, not the data scale. In practice, this model has all the same problems as the bug-fixed offset model, where the estimate of the mean is so large that we assign far too many values to NP.

1.3 Statistics jail

Now it's time to get unprincipled. In order to replicate SCEPTRE, we need to find a justification to do the following:

$$\text{use } \frac{\text{Pois}(y|e^o + \delta)}{\text{Pois}(y|e^o)} \quad \text{instead of} \quad \frac{\text{Pois}(y|e^o + e^\delta)}{\text{Pois}(y|e^o)}$$

In words, we need a reason to use the log of the estimated mean in our decision boundary instead of just using the mean.

I think there actually is a decent mathematical argument for using this. I'm still thinking through it, but basically it comes down to how e^δ is a very poor fit for a lot of our data points, and especially when things are heavy tailed. Since we just need a good decision boundary, using δ is actually pretty sane.

For cells with a very low NP mean e^{o_i} , we are effectively comparing to the constant distribution $\text{Pois}(\delta)$. Comparing to $\text{Pois}(e^\delta)$ performs very poorly, since our classification boundary is typically ≤ 20 , so these points are nowhere close to e^δ .

And for cells with a large e^{o_i} , it still performs fine.

I'm going to do more mathematical analysis once we discuss and confirm that this direction is useful.

2. Simulation results

In this section, I compare 6 models: SCEPTRE, crispat, and 4 custom ones. For the custom ones, the name is of the form $\langle x \rangle_{\langle y \rangle}$, with $\langle x \rangle$ denoting the offset method, and $\langle y \rangle$ denoting the mixture method.

`glmpois_poisumlog` is the additive model that uses δ instead of e^δ , and is the proposed replacement. The offsets are exactly the same as with SCEPTRE. Only the mixture is different.

The other 3 methods all use thresholding to fit the GLM only on the cells with $y_i \leq 1000$. They vary in their mixture:

- `threshglmpois1000_poisbug` is these robustified offsets with the current bugged Poisson-Pois mixture used by SCEPTRE
- `threshglmpois1000_poisumlog` is the additive model using the log mean, except also with robustified offsets
- `threshglmpois1000_pois0nb` is the robustified offsets with a Poisson-NB mixture, and represents the best method I found that actually uses $\exp(o_i + \gamma)$ as the perturbed mean.

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.2.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Reminder of my current simulation settings, copied from my report on Monday: inspired by Cellbender model, I simulate guides via

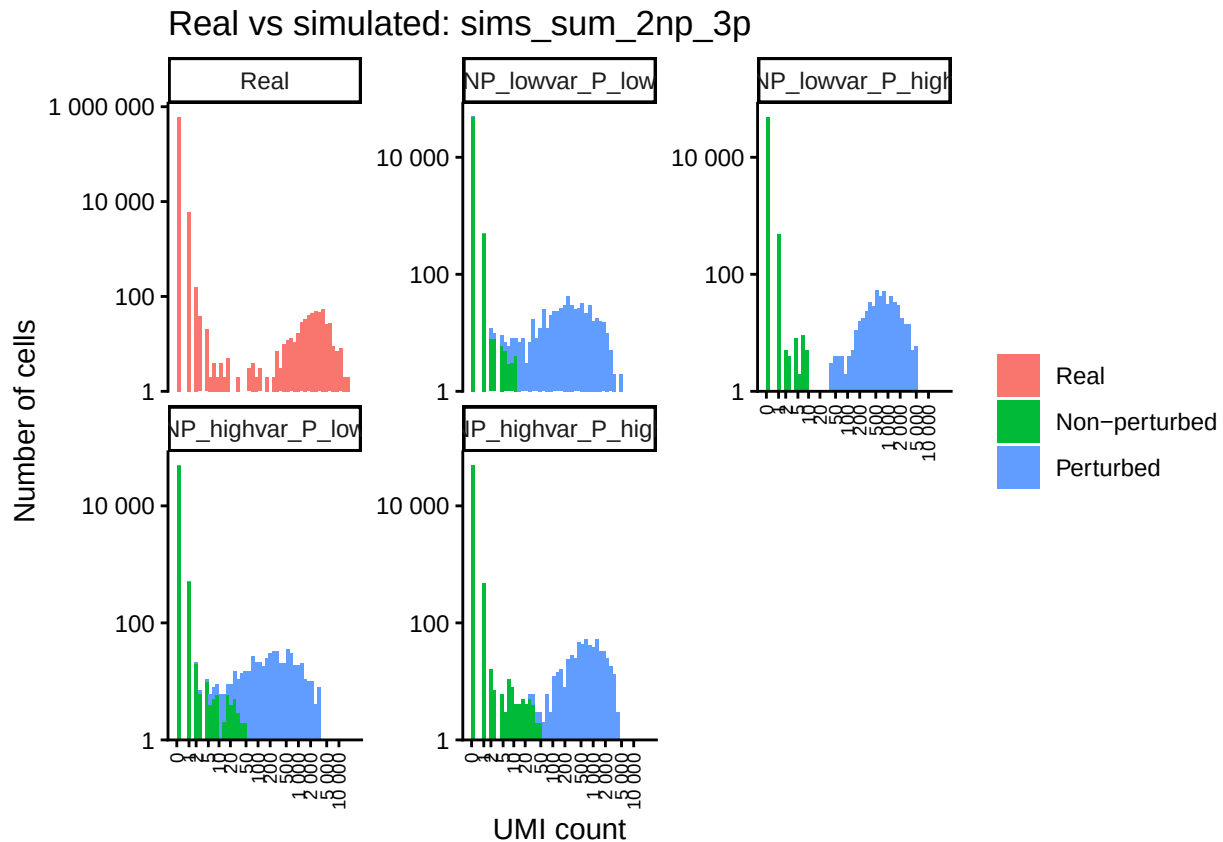
$$Y_i = \text{Pois}(L_i \mu_{\text{exo}}) + \text{Ber}(\pi_{\text{endo}}) \text{Pois}(L_i \mu_{\text{endo}}) + \text{Ber}(\pi_{\text{pert}}) \text{NB}(L_i \mu_{\text{pert}}, \theta_{\text{pert}}).$$

The idea:

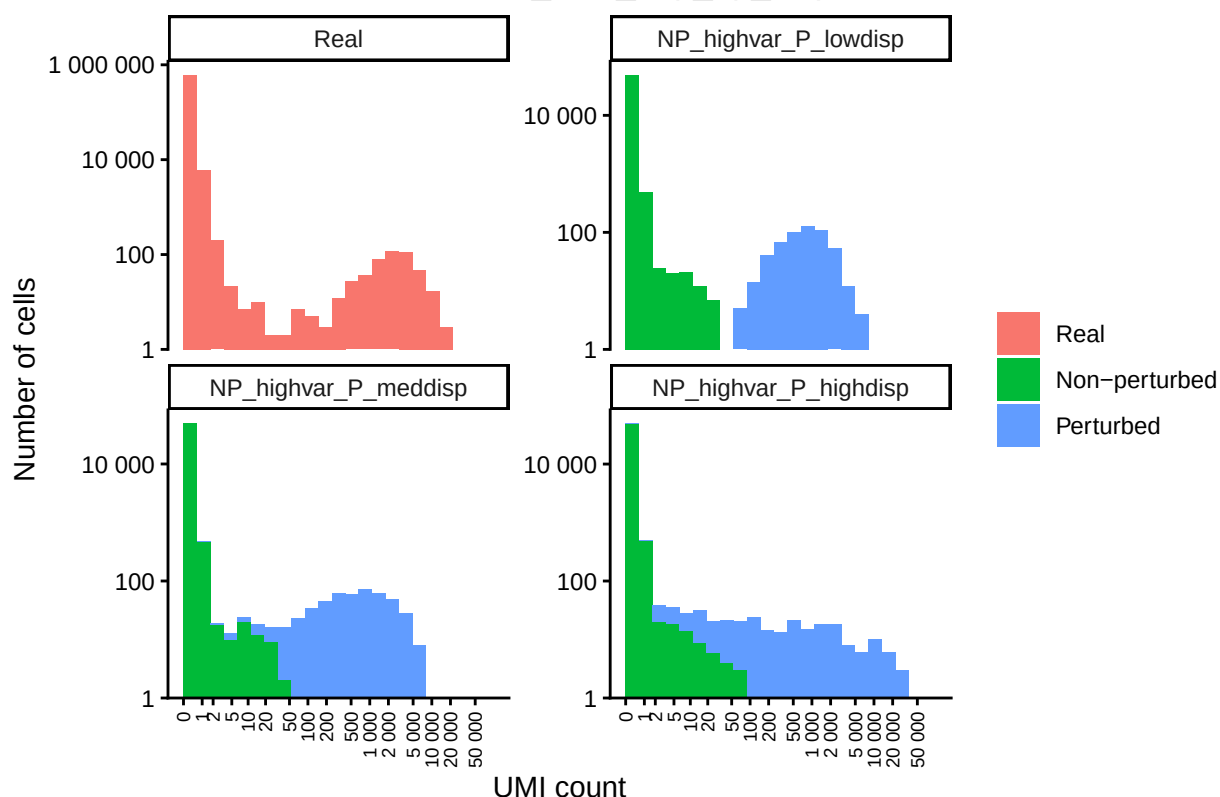
- the perturbed distribution is the same: NegBin with cell-specific library size rescaling factors
- but now the NP is a sum of two processes:
 - exogenous noise, representing random stray bits getting in there. μ_{exo} is small
 - endogenous noise, from droplet contamination, chimeras, etc, giving rare but larger values. π_{endo} is very small but μ_{endo} is larger.

I am able to get satisfying looking distributions from this, and in particular I can now simulate distributions where precision is the hard part. Before, with such light tails, precision was often trivial because the NPs were so concentrated near zero.

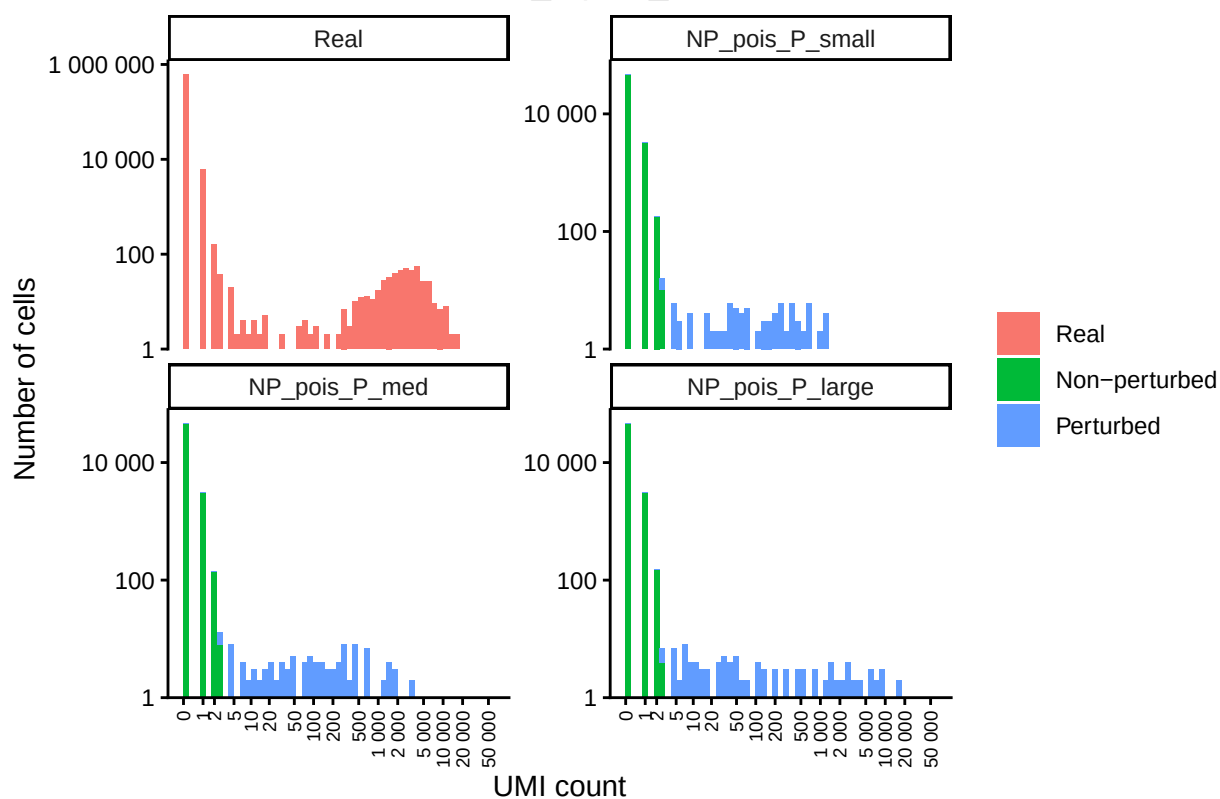
```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2     4.0.1      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr       1.2.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Loading required package: Matrix
##
##
## Attaching package: 'Matrix'
##
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```



Real vs simulated: sims_sum_1np_3p_disp



Real vs simulated: sims_repeat_old



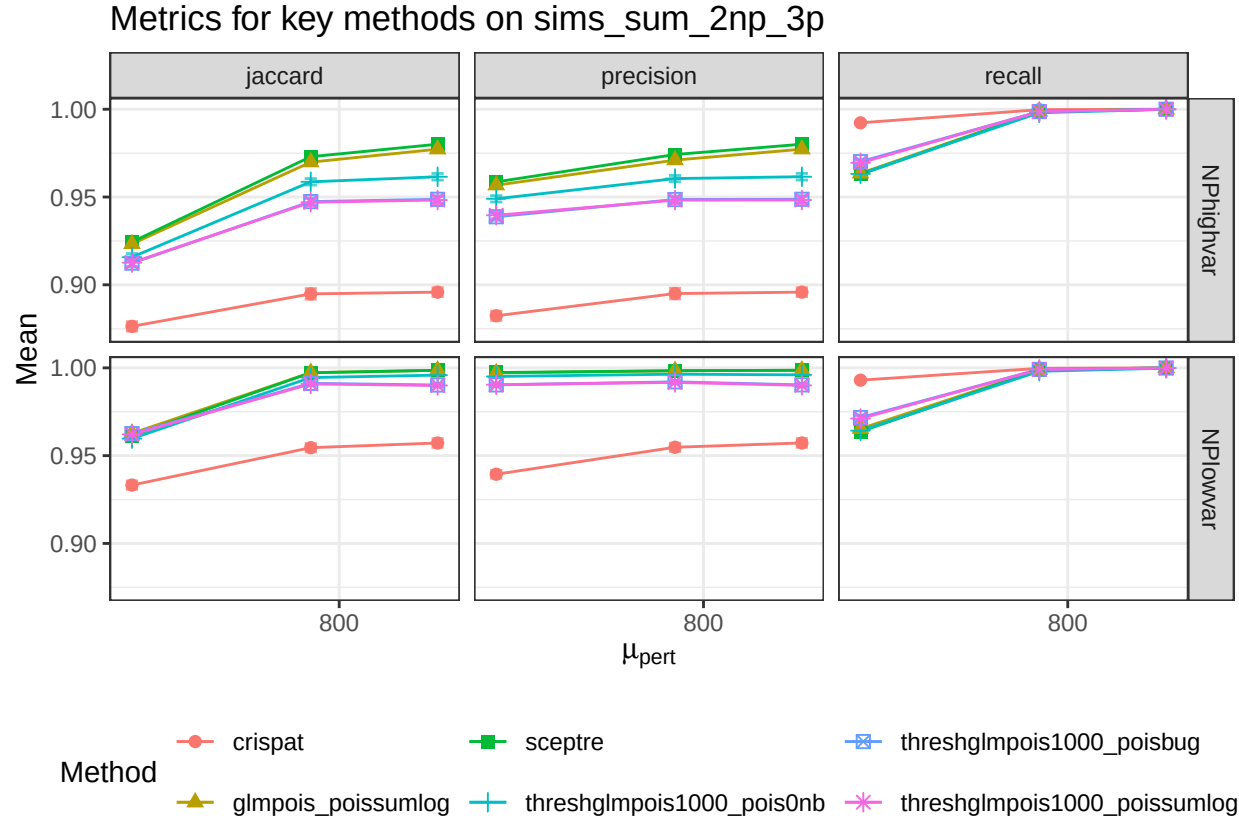


Figure 1: Comparing sceptre, crispat, and the new methods on data simulated with varying NP tails and perturbed means / overdispersions. SCEPTRE and `glm_pois_poissumlog` perform very similarly.

Metrics for more overdispersed data

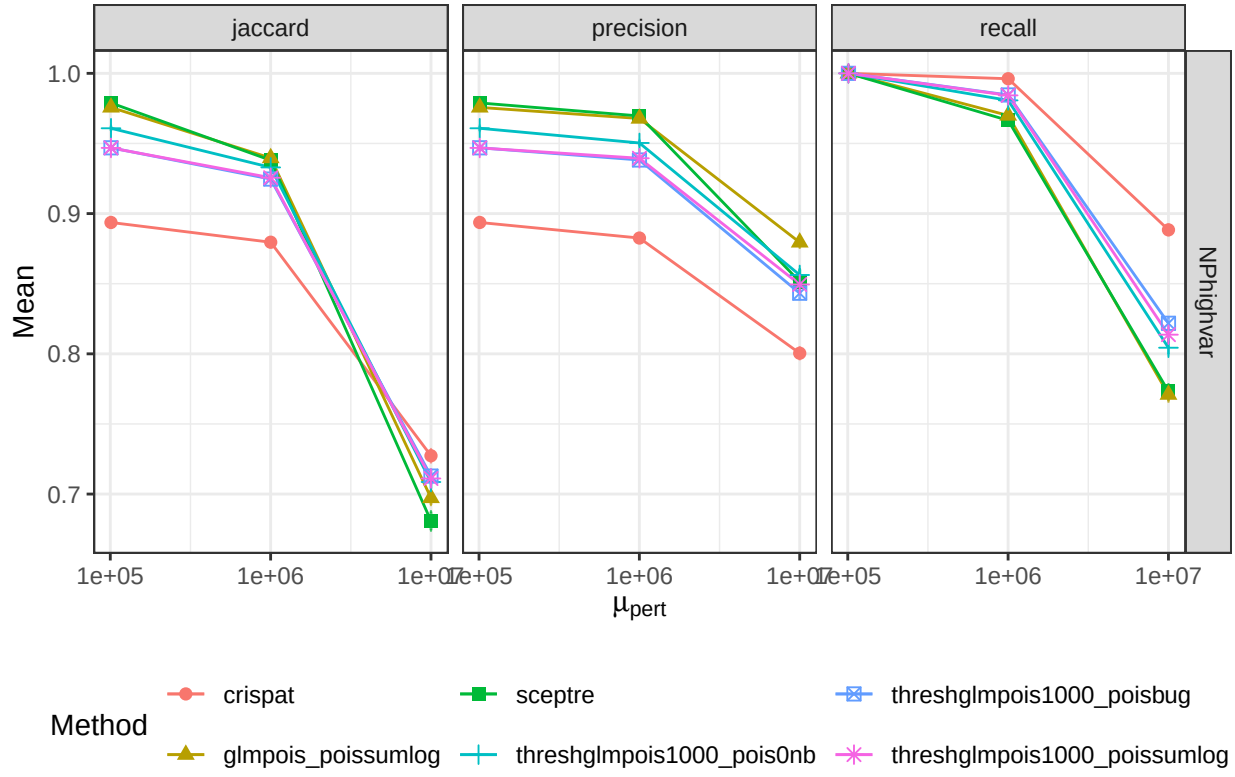


Figure 2: Simulations with much bigger and more overdispersed values. We see very similar performance between SCEPTRE and glmpois_poisumlog.

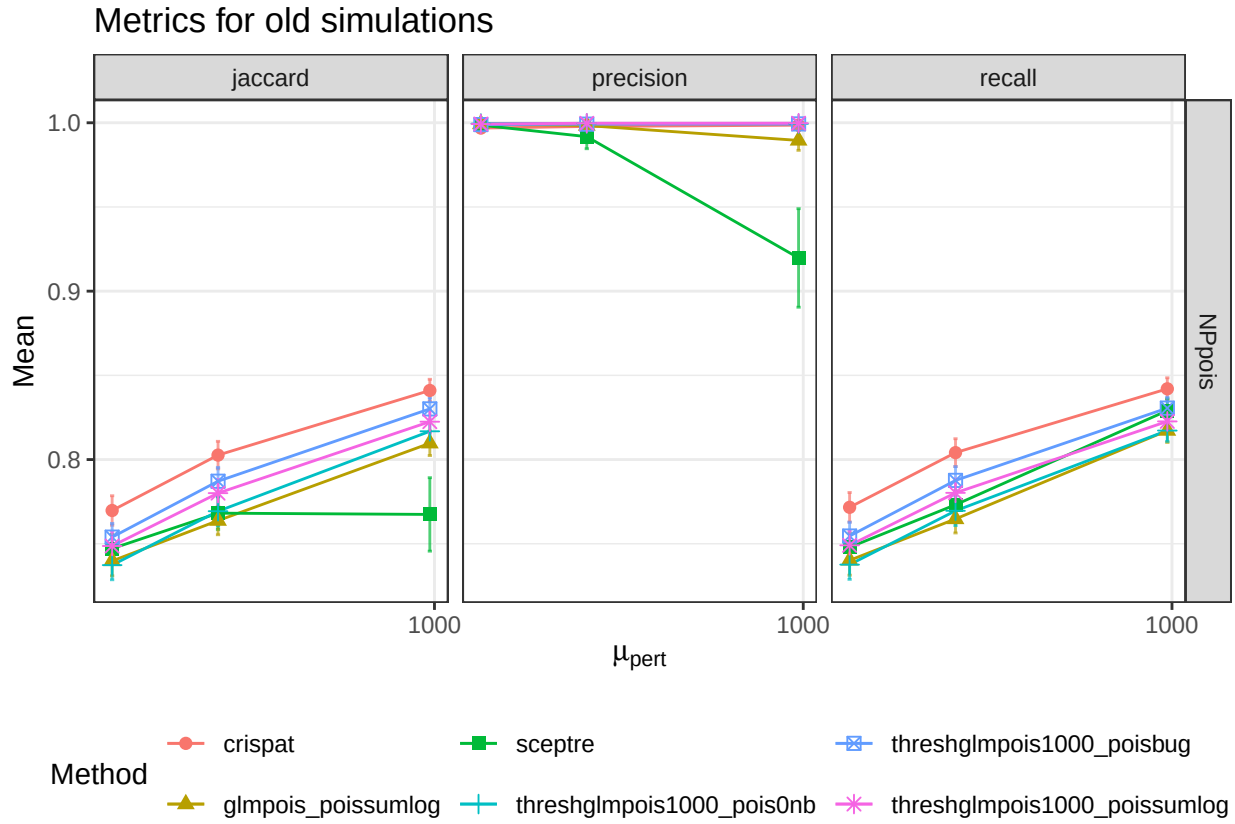


Figure 3: Repeating old simulations. This is a **HIGHLY** overdispersed pert and very low mean non-pert, so basically every non-pert is ≤ 3 , while the perts are all over the place. SCEPTRE breaks down here but the more principled estimation in glm_poisumlog keeps things ok.

3. Data results

I want to see how much my new methods change vs SCEPTRE. This plot computes precision and recall against SCEPTRE on Replogle.

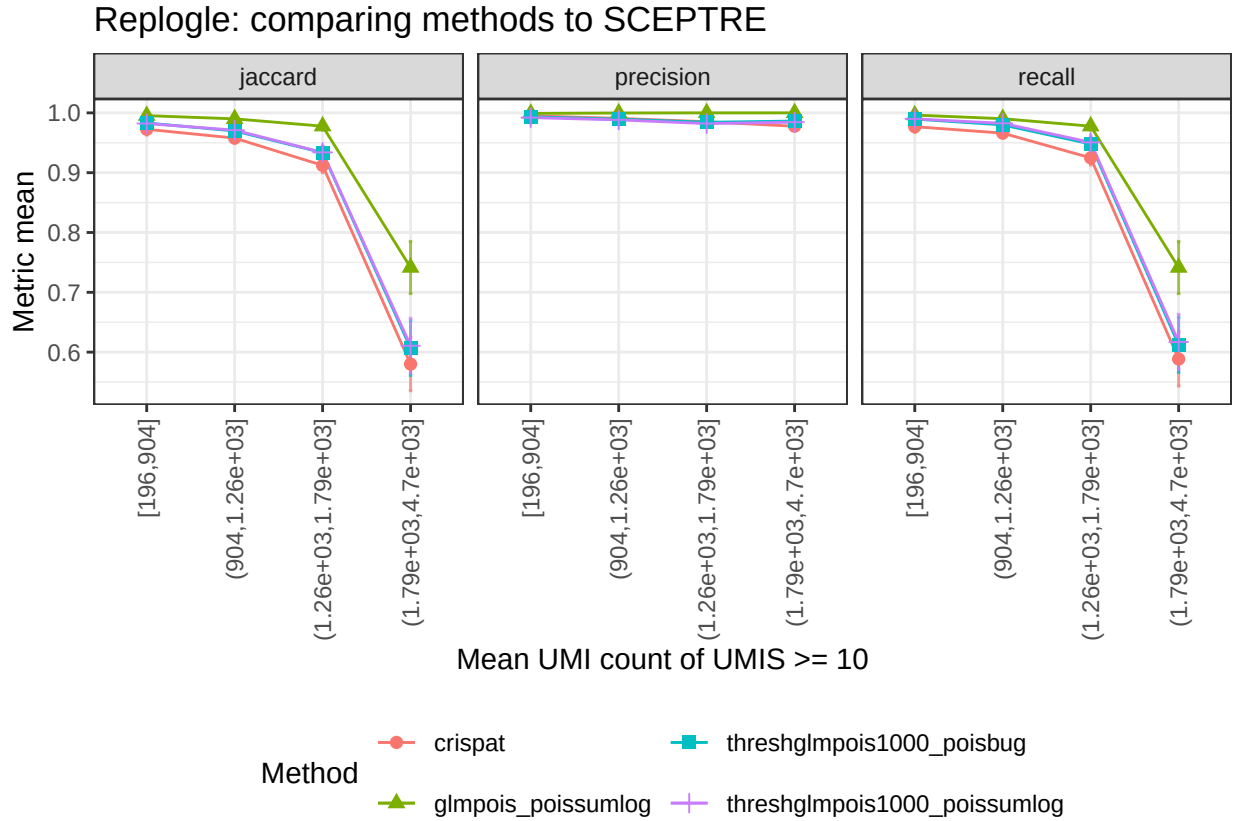


Figure 4: This plot bins Replogle guides based on `mean(guide[gudie >= 10])`, and computes the average metric in that group. SCEPTRE is the baseline. We see that our new `glm_poissumlog` is very close, until the heaviest tailed guides. The low recall of every method here vs SCEPTRE shows that SCEPTRE makes more positive predictions than they all do.

Scale for shape is already present.

Adding another scale for shape, which will replace the existing scale.

All methods perform very similarly on Gasperini so those plots are not interesting.

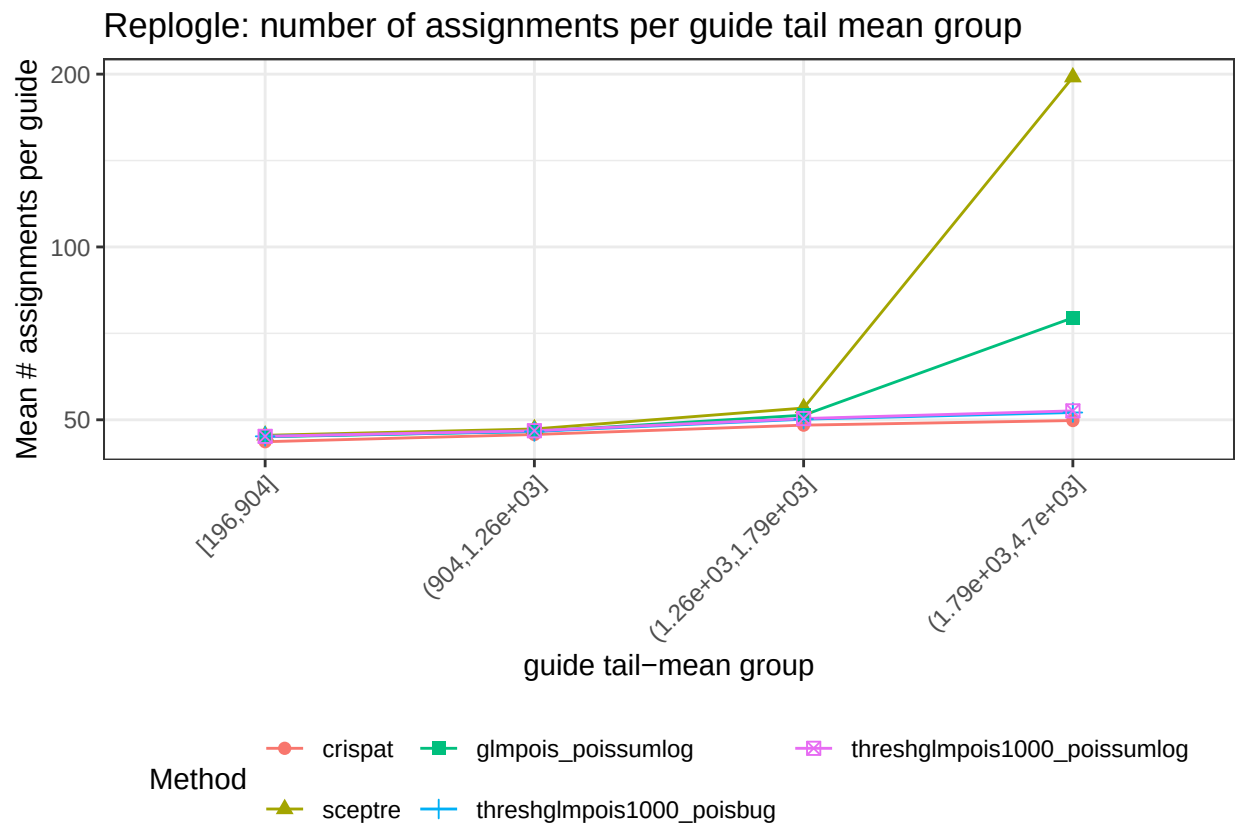


Figure 5: This plot shows that `glmpois_poissumlog` has some of the breakdown that SCEPTRE exhibits, in terms of making more assignments for heavier tailed guides, but nowhere nearly as bad.